TaskFlow

Plan • Organize • Achieve

Task Assignment & Roles

Personal & Collaborative Project Management Platform

PROGRAM	**DEPI**
DOCUMENT	**Task Assignment & Roles**
VERSION	**1.0**
PREPARED	**14 October 2026**
TEAM SIZE	**5 members**
AI	**Out of scope**

> **Public-page exclusion:** The public marketing website pages (such as Landing Page, Features / How It Works, Pricing, and optional Contact / FAQ pages) will be implemented separately by **Mostafa Mohamed Mahmoud Mohamed**. They are intentionally excluded from the internal application module distribution below and are not counted as a separate teammate's responsibility.

1. Purpose

This document defines ownership and responsibilities for the five TaskFlow team members. TaskFlow is designed for individuals, freelancers, teams, and companies. The responsibilities are organized by feature module so that each member owns their assigned user interface, backend logic, data changes, validation, and tests. Shared integrations are coordinated through agreed interfaces and GitHub pull requests.

2. Team Assignment Summary

Team member	Role	Primary module ownership
Mostafa Mohamed Mahmoud Mohamed	Team Lead; Authentication, Platform Foundation & Personal Productivity	Authentication, onboarding, personal dashboard, My Day / My Work, Quick Capture, global search, saved views, notification center, shared architecture and integration coordination
Ahmed Reda Ahmed Sedik	Workspace, Teams & Project Management	Workspaces, teams, members, invitations, membership roles, access rules, projects, project settings, templates
Alaa Essam Hamed Abdelkader	Task Management & Collaboration	Kanban, task lifecycle/details, subtasks, checklists, labels, comments, attachments, task activity and handover notes
Ahd Ammar Eisa El-Baqary	Time Tracking, Scheduling & Project Health	Timer, time entries/timesheets, calendar views, Gantt, dependencies, time reports and rule-based health alerts
Eman Mohamed Masad El-Metwally	Client Portal & Delivery Workflow	Client records/access, Project Live Room, client-facing progress, deliverables, review/approval, change requests

Ownership principle: Each member is responsible for the frontend pages, controller/service logic, relevant data model changes, validation, and tests for their assigned module. This is a full-stack feature split, not a split where some members only build frontend and others only build backend.

3. Mostafa Mohamed Mahmoud Mohamed — Team Lead, Authentication & Personal Productivity

3.1 Assigned pages and features

Authentication and account management

- Login and registration.
- Email verification, if included in the approved MVP.

- Forgot-password and reset-password flows.
- Initial onboarding and account setup.
- Profile, account preferences, security, and active-session pages where included in scope.

Personal Productivity & Navigation module

- Personal Dashboard: active projects, tasks due today, overdue tasks, recent updates, and quick links.
- My Day / My Work: a consolidated view of tasks across projects the user is authorized to access, grouped into Today, Upcoming, Overdue, and Completed.
- Quick Capture: a lightweight form or drawer for capturing a task from the dashboard or main navigation. Alaa owns the underlying task domain and task-creation business rules; Mostafa owns the quick-capture experience and its integration contract.
- Global Search: search across authorized projects, tasks, and other approved records. Search results must obey access rules owned by each module.
- Saved Views & Filters: users can save reusable personal filters, such as “My High-Priority Tasks” and “Due This Week.”
- Notification Center: notification list, read/unread status, basic filters, and notification preferences.

3.2 Backend and technical responsibilities

- Establish the shared solution structure: `TaskFlow.Web`, `TaskFlow.Services`, and `TaskFlow.Data`.
- Configure ASP.NET Core Identity and account flows.
- Define common authentication/authorization conventions with Ahmed, who owns workspace/project membership rules.
- Implement user preferences and persistence required by the Personal Productivity module.
- Define the shared notification service contract and base SignalR setup; module owners publish relevant events through the shared contract instead of creating separate notification systems.
- Coordinate access-aware global search with the other module owners. Each module remains responsible for enforcing server-side authorization on its own data.
- Define common error handling, logging, configuration, and secret-management conventions.
- Coordinate repository rules, feature branches, pull requests, integration, release checklist, and deployment preparation.
- Coordinate EF Core migration integration to reduce conflicts; module owners must communicate schema changes before shared migrations are merged.

3.3 Team-lead responsibilities

- Track milestones, blockers, and scope changes.
- Coordinate shared contracts and resolve cross-module dependencies.
- Review integration readiness and ensure each feature has acceptance criteria and test evidence.
- Coordinate end-to-end testing, final GitHub documentation, and demonstration preparation.

3.4 Boundaries with other modules

- Ahmed owns workspace/project records, memberships, invitations, and access configuration.
- Alaa owns the authoritative task model and task-creation/edit business rules.
- Ahd owns timer records, Gantt, calendar, time reports, and project-health alert rules.
- Eman owns client data, client-visible access, deliverables, review decisions, and change requests.
- Mostafa's dashboard, search, saved views, and Quick Capture consume agreed services and must not duplicate the business logic owned by other members.

Separate task: Mostafa also develops the public marketing pages identified at the top of this document. That work is outside the shared application-module allocation.

4. Ahmed Reda Ahmed Sedik — Workspace, Teams & Project Management

4.1 Assigned pages and features

- Workspace list / switcher data and workspace overview.
- Create/edit workspace and workspace settings.
- Workspace projects list.
- Create/edit project and project settings.
- Project overview and project template selection/basic templates.
- Teams list and team details/membership.
- Workspace members and invitation management.
- Role/access settings for workspace and project membership.
- Project members and access configuration.

4.2 Backend and data responsibilities

- Own workspace, team, membership, project, and invitation entities, such as `Workspace`, `WorkspaceMember`, `Team`, `TeamMember`, `Project`, `ProjectMember`, `Invitation`, and `ProjectTemplate` (final names subject to the agreed ERD).
- Implement services/controllers for workspace and project creation/update, memberships, invitations, and project configuration.
- Enforce workspace and project access rules on the server.
- Coordinate with Mostafa on identity/account contracts, Alaa on boards/columns/tasks, and Eman on client access to projects.
- Test membership changes, invitations, project permissions, workspace isolation, and unauthorized direct access.

5. Alaa Essam Hamed Abdelkader — Task Management & Collaboration

5.1 Assigned pages and features

- Kanban board and task-card interactions.
- Create/edit task form and Task Details view.
- Task status transitions, ordering, priority, labels, due dates, time estimates, and assignees.
- Subtasks and checklist management.
- Comments and discussions, including @mentions UI.
- Attachment UI and authorized upload/download workflow.
- Task activity history.
- Work Handover notes and handover history.

- Board-column interactions, WIP limits, and swimlanes if retained in the agreed scope.

5.2 Backend and data responsibilities

- Own the task workflow and related entities, such as `Board`, `BoardColumn`, `TaskItem`, `TaskAssignee`, `Subtask`, `ChecklistItem`, `Label`, `TaskLabel`, `Comment`, attachment metadata, `TaskActivity`, and `HandoverNote`.
- Implement task creation, updates, assignment, ordering, status-transition rules, validation, and archive behavior.
- Coordinate with Mostafa's shared notification/SignalR interface rather than creating a separate notification mechanism.
- Coordinate stable Project/Task identifiers and service contracts with Ahmed, Ahd, and Eman.
- Test task operations, state changes, ordering, comments, attachment access, subtasks/checklists, and handover records.

6. Ahd Ammar Eisa El-Baqary — Time Tracking, Scheduling & Project Health

6.1 Assigned pages and features

- Persistent/global timer control: Start, Pause, Resume, and Stop.
- My Time, time entries, and timesheet pages.
- Time-entry details and correction workflow, if retained in scope.
- Project time report and estimated-versus-actual time view.
- My Calendar and Project Calendar.
- Gantt chart, milestones, timeline, and task-dependency view.
- Project Health dashboard.
- Health-alert list, alert details, and threshold settings.

6.2 Backend and data responsibilities

- Own timer/schedule entities such as `TimeEntry` or `TimerSession`, `TaskDependency`, `Milestone` (if included), and `ProjectHealthAlert`.
- Implement start/pause/resume/stop with authoritative server timestamps and prevent duplicate active sessions according to the agreed rule.
- Calculate totals by user, task, project, and reporting period.
- Implement schedule/dependency validation within the agreed scope, including prevention of invalid circular dependencies if supported.
- Implement deterministic, rule-based alerts for agreed conditions, such as overdue critical tasks, long-running blockers, exceeded time estimates, and deliverables awaiting review.
- Coordinate with Alaa for task/project references and with Eman for review/deliverable state contracts used by project-health alerts.
- Test timer lifecycle, persistence, duration totals, duplicate sessions, dependency behavior, reporting values, and alert thresholds.

7. Eman Mohamed Masad El-Metwally — Client Portal & Delivery Workflow

7.1 Assigned pages and features

- Clients list and client profile.
- Invite client and client access configuration.
- Client portal dashboard and client project list.
- Project Live Room.
- Client-facing project progress, shared milestones, approved activity, and shared discussions.
- Internal deliverables list and create/upload deliverable flow.
- Deliverable details, versions, review queue, and review history.
- Client review actions: approve, request changes, and submit feedback.
- Change-request list, create request, details, impact fields, decision, and history.

7.2 Backend and data responsibilities

- Own the client/delivery entities, such as `Client`, `ClientProjectAccess`, `Deliverable`, `DeliverableVersion`, `DeliverableReview`, and `ChangeRequest`.
- Enforce strict client data isolation: clients can access only projects, files, reports, and conversations explicitly shared with them.
- Record each review against a specific deliverable version, including reviewer, decision, timestamp, and feedback.
- Record requested changes and status; do not silently alter agreed project scope before authorized approval.
- Coordinate with Ahmed on project membership/access, Alaa on tasks/comments/attachments, and Ahd on client-visible time/progress summaries.
- Test client permissions, direct URL/API access, deliverable visibility, review decisions, and the change-request lifecycle.

8. Shared Responsibilities and Dependencies

Area	Primary owner	Required collaboration
Solution architecture and repository conventions	Mostafa	All members agree on the structure and shared contracts before major changes
Authentication and account lifecycle	Mostafa	Ahmed coordinates membership/invitation rules; Eman coordinates client-account access
Workspace, project, and membership rules	Ahmed	All modules use and respect agreed access rules
Personal Dashboard, My Work, Quick Capture, Saved Views	Mostafa	Alaa supplies task contracts; Ahmed supplies project/workspace access rules
Task and board data	Alaa	Ahd consumes task references for timer/Gantt; Eman links deliverables and requests to projects/tasks
Timer and time reports	Ahd	Eman only exposes time summaries that are explicitly approved for client viewing

Area	Primary owner	Required collaboration
Client-visible data boundary	Eman	Ahmed confirms project access; all module owners enforce visibility rules
Shared notifications and SignalR	Mostafa	Each feature owner emits agreed events; access is checked before sending updates
EF Core migrations integration	Mostafa coordinates	Entity owners communicate schema changes and avoid conflicting migrations
Testing	All members	Every member tests their module; the full team tests end-to-end journeys
Deployment and release	Mostafa coordinates	Each owner fixes defects in their own module
Documentation and final presentation	All members	Each member documents their module; Mostafa coordinates consistency

9. Required End-to-End Test Scenarios

1. Register/login → create personal workspace → open My Day → capture a task quickly → open it in Kanban and move it through the workflow.
2. Search for a task across authorized projects and save a personal filter; confirm search never reveals records from inaccessible projects.
3. Start timer on a task → pause/resume/stop → confirm the saved time entry and project report.
4. Create a shared workspace → invite a member → confirm role and project-access boundaries.
5. Invite a client → client opens only a shared project → team shares a deliverable → client approves it or requests changes.
6. Create a change request → assess impact → approve/reject → confirm the decision is recorded and the approved change is reflected in the plan/backlog as agreed.
7. Hand over a task to another member → confirm notes and history remain linked to the task.
8. Trigger a configured project-health rule → confirm the alert is generated for the appropriate authorized user.

10. GitHub Workflow

- Use a feature branch per coherent feature, for example `feature/authentication`, `feature/workspace-management`, `feature/task-kanban`, `feature/time-tracking`, or `feature/client-review`.
- Avoid multiple members editing the same EF Core migration or shared entity contract concurrently. Agree on schema changes before integration.
- Merge through Pull Requests; include a summary and test evidence, and screenshots for UI changes where useful.
- Do not commit secrets, credential-containing connection strings, or real client data.
- Update the relevant documentation when behavior or scope changes.

11. Definition of Done

A task is complete when its acceptance criteria are met, required server-side authorization is enforced, data persists correctly, important success and failure cases are tested, relevant documentation is updated, and changes are reviewed and merged through GitHub.

12. Team Confirmation

This is the proposed responsibility split based on the agreed TaskFlow scope. The team should confirm assignments against each member's availability and experience before implementation begins. Any change should be recorded in the project documentation so ownership remains clear.